



MASTG

- MASTG-TEST-0200: Files Written to External Storage
- MASTG-TEST-0201: Runtime Use of APIs to Access External Storage
- MASTG-TEST-0202: References to APIs and Permissions for Accessing External Storage
- MASTG-TEST-0203: Runtime Use of Logging APIs
- MASTG-TEST-0207: Runtime Storage of Unencrypted Data in the App Sandbox
- MASTG-TEST-0216: Sensitive Data Not Excluded From Backup
- MASTG-TEST-0231: References to Logging

L1

L2

P

ANDROID

MASWE-0001

TEST



MASTG-TEST-0203: Runtime Use of Logging APIs

Overview

On Android platforms, logging APIs like `Log` , `Logger` , `System.out.print` , `System.err.print` , and `java.lang.Throwable#printStackTrace` can inadvertently lead to the leakage of sensitive information. Log messages are recorded in logcat, a shared memory buffer, accessible since Android 4.1 (API level 16) only to privileged system applications that declare the `READ_LOGS` permission. Nonetheless, the vast ecosystem of Android devices includes pre-loaded apps with the `READ_LOGS` privilege, increasing the risk of sensitive data exposure. Therefore, direct logging to logcat is generally advised against due to its susceptibility to data leaks.

Steps

1. Install and run the app.
2. Navigate to the screen of the mobile app you want to analyse the log output from.
3. Execute a method trace ([🔗 Method Tracing](#)) (using e.g. [🔗 Frida \(Android\)](#)) by attaching to the running app, targeting logging APIs and save the output.

Observation

The output should contain a list of locations where logging APIs are used in the app for the current execution.

Evaluation

The test case fails if you can find sensitive data being logged using those APIs.

Best Practices

🤖 MASTG-BEST-0002: Remove Logging Code

Demos

🤖 MASTG-DEMO-0006: Tracing Common Logging APIs Looking for Secrets